

parallel merge explanation

13 May 2026 01:46

HPC Assignment 4 — Parallel Merge Sort using OpenMP

This assignment implements:

Sequential and Parallel Merge Sort

using:

- divide and conquer
- recursion
- OpenMP parallel sections

1. What We Have To Do?

We have to:

1. input array elements
2. perform sequential merge sort
3. perform parallel merge sort
4. compare execution times

Main Goal

To demonstrate:

parallel divide-and-conquer sorting

using OpenMP.

2. What is Merge Sort?

Merge Sort is:

divide and conquer sorting algorithm

Steps:

1. divide array into halves
2. sort each half recursively
3. merge sorted halves

Example

Array:

5 2 4 1

Divide

[5 2] [4 1]

Divide Again

[5] [2] [4] [1]

Merge

[2 5] [1 4]

Final Merge

[1 2 4 5]

Why Merge Sort Efficient?

Because:

- divides problem recursively
- merging is efficient

Time Complexity

Sequential Merge Sort:

$O(n \log n)$

Better than bubble sort.

3. Overall Workflow

Input Array



Divide Into Halves



Recursively Sort Halves



Merge Sorted Halves



Final Sorted Array

4. Merge Function

Code:

```
void merge(vector<int>& arr, int left, int mid, int right)
```

MOST IMPORTANT FUNCTION.

Purpose

Merges:

two already sorted halves

into one sorted array.

Example

Left half:

1 4 7

Right half:

2 3 8

After merge:

1 2 3 4 7 8

Left and Right Sizes

```
int n1 = mid - left + 1;
```

```
int n2 = right - mid;
```

Calculates sizes of:

- left subarray
- right subarray

Temporary Arrays

```
vector<int> L(n1), R(n2);
```

Stores divided halves temporarily.

Copy Data

```
L[i] = arr[left + i];
```

```
R[j] = arr[mid + 1 + j];
```

Copies original array into temporary arrays.

Merge Logic

```
while (i < n1 && j < n2)
```

Compare:

- left element
- right element

Smaller element inserted first.

Example

Compare:

1 and 2

Insert:

1

Then compare next elements.

Remaining Elements

After one array finishes:

```
while (i < n1)
```

```
while (j < n2)
```

Copies remaining elements.

5. Sequential Merge Sort

Code:

```
void sequentialMergeSort(vector<int>& arr, int left, int right)
```

Base Condition

```
if (left >= right)
```

```
    return;
```

Stops recursion when:

only one element remains

because single element already sorted.

Find Mid

```
int mid = left + (right - left) / 2;
```

Splits array into halves.

Recursive Calls

```
sequentialMergeSort(arr, left, mid);
```

```
sequentialMergeSort(arr, mid + 1, right);
```

Sort:

- left half

- right half

Merge Halves

`merge(arr, left, mid, right);`
Combines sorted halves.

6. Parallel Merge Sort

MOST IMPORTANT PART.

Code:

`void parallelMergeSort(vector<int>& arr, int left, int right, int depth)`

Why Parallel Merge Sort Possible?

Because:

left and right halves are independent

They can be sorted simultaneously.

Example

Array:

[5 2 4 1]

Thread 1 sorts:

[5 2]

Thread 2 sorts:

[4 1]

at same time.

7. Depth Parameter

`int depth`

Controls:

recursion parallelism level

Why Needed?

Too many threads create:

- overhead
- inefficiency

Depth limits thread creation.

Important Logic

`if (depth <= 0)`

When depth becomes 0:

switch to sequential merge sort

Why?

To reduce:

excessive parallel overhead

8. OpenMP Parallel Sections

MOST IMPORTANT CONCEPT.
#pragma omp parallel sections

Purpose

Allows:

different code sections to run simultaneously

Section 1

```
#pragma omp section
parallelMergeSort(arr, left, mid, depth - 1);
```

Sorts left half.

Section 2

```
#pragma omp section
parallelMergeSort(arr, mid + 1, right, depth - 1);
```

Sorts right half.

Both Run Simultaneously

Using:

separate threads

After Both Complete

```
merge(arr, left, mid, right);
```

Final merging happens.

9. Main Function

```
int main()
```

Program starts here.

Input Array

```
vector<int> arr(n), arr_copy;
```

Two copies:

- sequential sorting
- parallel sorting

Why Copy Needed?

To compare:

same input data

fairly.

10. Timing Sequential Execution

```
double start = omp_get_wtime();
```

Records start time.

Sequential Sort

```
sequentialMergeSort(arr, 0, n - 1);
```

End Time

`double end = omp_get_wtime();`

Total Time

(end - start)

Execution duration.

11. Parallel Execution

`parallelMergeSort(arr_copy, 0, n - 1, 3);`

Depth:

3

can be adjusted.

Higher depth:

- more threads
- more parallelism
- more overhead

12. Example Execution

Input:

5 2 4 1 3

Divide

[5 2] [4 1 3]

Sort Halves

[2 5] [1 3 4]

Final Merge

1 2 3 4 5

13. Time Complexity

Sequential Merge Sort:

$O(n \log n)$

Parallel Merge Sort

Approximate:

$O\left(\frac{n \log n}{P}\right)$

Where:

- (P) = processors/threads

Why Faster?

Because:

- recursive halves sorted simultaneously

14. Advantages of Merge Sort

Advantage

Explanation

Faster than bubble sort	better complexity
Parallelizable	halves independent
Stable sort	preserves order

15. Important Viva Questions

Q1. What is Merge Sort?

Divide-and-conquer sorting algorithm.

Q2. Why Merge Sort efficient?

Because time complexity is:

$O(n \log n)$

Q3. What is divide and conquer?

Breaking problem into smaller subproblems and combining solutions.

Q4. Why Merge Sort suitable for parallelism?

Because left and right halves can be processed independently.

Q5. What does #pragma omp parallel sections do?

Runs multiple code sections simultaneously.

Q6. Why depth parameter used?

To limit thread creation overhead.

Q7. What is base condition in recursion?

if(left >= right)

Q8. Why temporary arrays used in merge?

To store left and right sorted halves.

16. Short Practical Explanation

This program implements sequential and parallel merge sort using OpenMP. Merge sort recursively divides the array into halves, sorts them, and merges sorted halves. Parallelism is achieved using OpenMP parallel sections where left and right recursive halves are processed simultaneously. Execution times of sequential and parallel merge sort are compared using `omp_get_wtime()`.